Name: Saai Vardhan D
Registration number : 192525205

CSA 0421 - OPERATING SYSTEM

Experiment 11

Illustrate the concept of multithreading using a C program.

Executable C code :

```
#include <stdio.h>
#include <pthread.h>

long result[3];

void *sum_thread(void *arg) {
    int n = *(int *)arg, i;
    long s = 0;
    for (i = 1; i <= n; i++) s += i;
    result[0] = s;
    return NULL;
}

void *factorial_thread(void *arg) {
    int n = *(int *)arg, i;
    long f = 1;
    for (i = 2; i <= n; i++) f *= i;
    result[1] = f;
    return NULL;
}

void *square_thread(void *arg) {
    int n = *(int *)arg;
    result[2] = (long)n * n;
    return NULL;
}

int main() {
    pthread_t t1, t2, t3;
    int n;

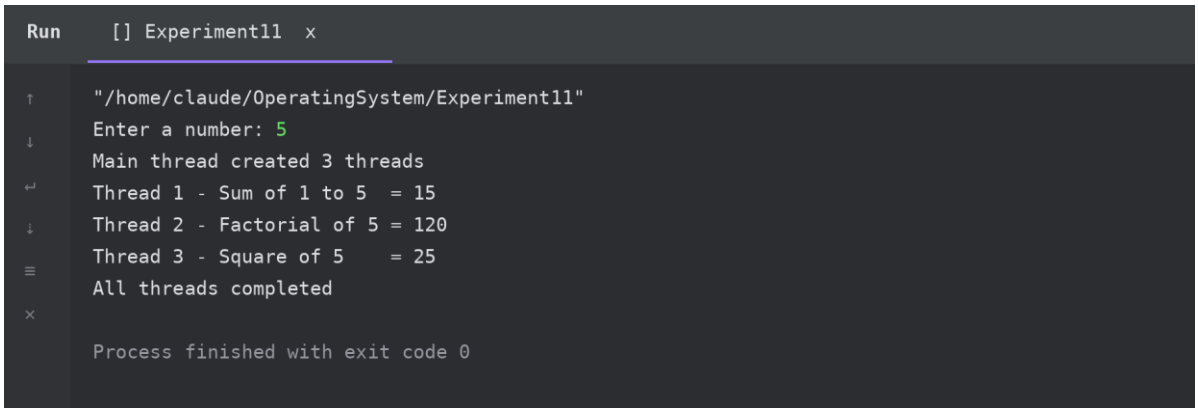
    printf("Enter a number: ");
    scanf("%d", &n);

    // create three threads that work on the same number at the same time
    pthread_create(&t1, NULL, sum_thread, &n);
    pthread_create(&t2, NULL, factorial_thread, &n);
    pthread_create(&t3, NULL, square_thread, &n);
    printf("Main thread created 3 threads\n");

    // wait for all threads to finish
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
    pthread_join(t3, NULL);

    printf("Thread 1 - Sum of 1 to %d  = %ld\n", n, result[0]);
    printf("Thread 2 - Factorial of %d = %ld\n", n, result[1]);
    printf("Thread 3 - Square of %d    = %ld\n", n, result[2]);
    printf("All threads completed\n");
    return 0;
}
```

Output Screenshot:



Experiment 12

Design a C program to simulate the concept of Dining-Philosophers problem

Executable C code :

```
#include <stdio.h>

int n;

// two philosophers sitting next to each other cannot eat together (they share a
fork)
int adjacent(int a, int b) {
    int d = a > b ? a - b : b - a;
    return d == 1 || d == n - 1;
}

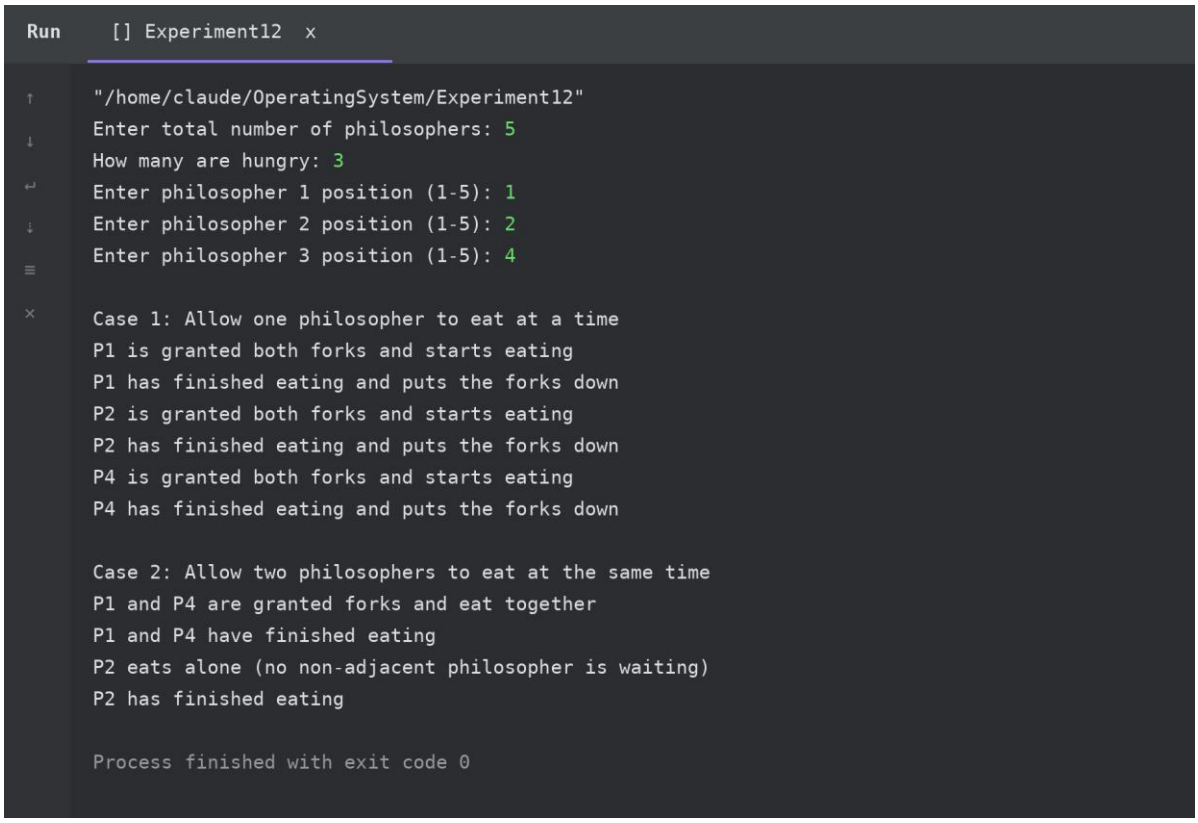
int main() {
    int h, i, j, hungry[10], served[10] = {0}, remaining;

    printf("Enter total number of philosophers: ");
    scanf("%d", &n);
    printf("How many are hungry: ");
    scanf("%d", &h);
    for (i = 0; i < h; i++) {
        printf("Enter philosopher %d position (1-%d): ", i + 1, n);
        scanf("%d", &hungry[i]);
    }

    printf("\nCase 1: Allow one philosopher to eat at a time\n");
    for (i = 0; i < h; i++) {
        printf("P%d is granted both forks and starts eating\n", hungry[i]);
        printf("P%d has finished eating and puts the forks down\n", hungry[i]);
    }

    printf("\nCase 2: Allow two philosophers to eat at the same time\n");
    remaining = h;
    while (remaining > 0) {
        for (i = 0; i < h && served[i]; i++);      // first philosopher still
waiting
        served[i] = 1;
        remaining--;
        for (j = i + 1; j < h; j++) {
            if (!served[j] && !adjacent(hungry[i], hungry[j])) break;
        }
        if (j < h) {
            served[j] = 1;
            remaining--;
            printf("P%d and P%d are granted forks and eat together\n", hungry[i],
hungry[j]);
            printf("P%d and P%d have finished eating\n", hungry[i], hungry[j]);
        } else {
            printf("P%d eats alone (no non-adjacent philosopher is waiting)\n",
hungry[i]);
            printf("P%d has finished eating\n", hungry[i]);
        }
    }
    return 0;
}
```

Output Screenshot:



Experiment 13

Construct a C program for implementation the various memory allocation strategies.

Executable C code :

```
#include <stdio.h>

int b[10], p[10], nb, np;

// type 1 = First Fit, 2 = Best Fit, 3 = Worst Fit
void allocate(int type, const char *name) {
    int bs[10], i, j, idx;

    for (i = 0; i < nb; i++) bs[i] = b[i];

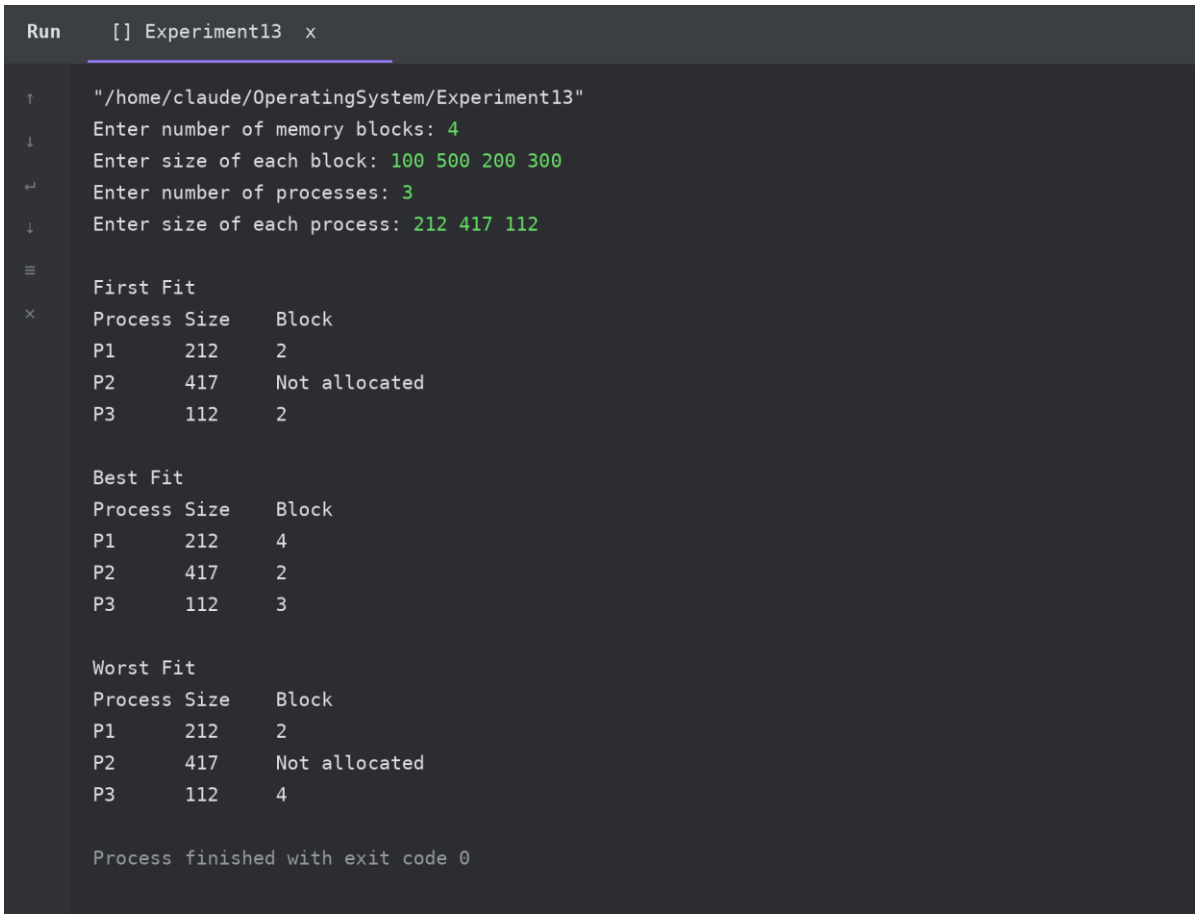
    printf("\n%s\n", name);
    printf("Process\tSize\tBlock\n");
    for (i = 0; i < np; i++) {
        idx = -1;
        for (j = 0; j < nb; j++) {
            if (bs[j] >= p[i]) {
                if (type == 1) { idx = j; break; }
                if (type == 2 && (idx == -1 || bs[j] < bs[idx])) idx = j;
                if (type == 3 && (idx == -1 || bs[j] > bs[idx])) idx = j;
            }
        }
        if (idx != -1) {
            printf("P%d\t%d\t%d\n", i + 1, p[i], idx + 1);
            bs[idx] -= p[i];
        } else {
            printf("P%d\t%d\tNot allocated\n", i + 1, p[i]);
        }
    }
}

int main() {
    int i;

    printf("Enter number of memory blocks: ");
    scanf("%d", &nb);
    printf("Enter size of each block: ");
    for (i = 0; i < nb; i++) scanf("%d", &b[i]);
    printf("Enter number of processes: ");
    scanf("%d", &np);
    printf("Enter size of each process: ");
    for (i = 0; i < np; i++) scanf("%d", &p[i]);

    allocate(1, "First Fit");
    allocate(2, "Best Fit");
    allocate(3, "Worst Fit");
    return 0;
}
```

Output Screenshot:



Experiment 14

Construct a C program to organize the file using single level directory.

Executable C code :

```
#include <stdio.h>
#include <string.h>

int main() {
    char dir[20][30], name[30];
    int count = 0, choice, i, found;

    do {
        printf("\n1. Create file  2. Delete file  3. Search file  4. Display
files  5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
        case 1:
            printf("Enter file name: ");
            scanf("%s", name);
            found = 0;
            for (i = 0; i < count; i++)
                if (strcmp(dir[i], name) == 0) found = 1;
            if (found)
                printf("File %s already exists\n", name);
            else if (count == 20)
                printf("Directory is full\n");
            else {
                strcpy(dir[count++], name);
                printf("File %s created\n", name);
            }
            break;
        case 2:
            printf("Enter file name to delete: ");
            scanf("%s", name);
            found = 0;
            for (i = 0; i < count; i++) {
                if (strcmp(dir[i], name) == 0) {
                    for (; i < count - 1; i++) strcpy(dir[i], dir[i + 1]);
                    count--;
                    found = 1;
                    break;
                }
            }
            printf(found ? "File %s deleted\n" : "File %s not found\n", name);
            break;
        case 3:
            printf("Enter file name to search: ");
            scanf("%s", name);
            found = 0;
            for (i = 0; i < count; i++)
                if (strcmp(dir[i], name) == 0) found = 1;
            printf(found ? "File %s found\n" : "File %s not found\n", name);
            break;
        case 4:
            if (count == 0)
                printf("Directory is empty\n");
            else {
                printf("Files in the root directory:\n");
                for (i = 0; i < count; i++) printf("  %s\n", dir[i]);
            }
            break;
        case 5:
            break;
        default:
            printf("Invalid choice\n");
        }
    } while (choice != 5);
```

```
    return 0;
}
```

Output Screenshot:



Experiment 15

Design a C program to organize the file using two level directory structure.

Executable C code :

```
#include <stdio.h>
#include <string.h>

struct directory {
    char dname[20];
    char files[10][20];
    int fcount;
} d[5];

int dcount = 0;

int find_dir(char *name) {
    int i;
    for (i = 0; i < dcount; i++)
        if (strcmp(d[i].dname, name) == 0) return i;
    return -1;
}

int find_file(int di, char *name) {
    int i;
    for (i = 0; i < d[di].fcount; i++)
        if (strcmp(d[di].files[i], name) == 0) return i;
    return -1;
}

int main() {
    int choice, di, fi, i, j;
    char dname[20], fname[20];

    do {
        printf("\n1. Create directory  2. Create file  3. Delete file\n");
        printf("4. Search file  5. Display  6. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch (choice) {
        case 1:
            printf("Enter directory name: ");
            scanf("%s", dname);
            if (find_dir(dname) != -1)
                printf("Directory %s already exists\n", dname);
            else if (dcount == 5)
                printf("Maximum directories reached\n");
            else {
                strcpy(d[dcount].dname, dname);
                d[dcount++].fcount = 0;
                printf("Directory %s created\n", dname);
            }
            break;
        case 2:
            printf("Enter directory name and file name: ");
            scanf("%s %s", dname, fname);
            di = find_dir(dname);
            if (di == -1)
                printf("Directory %s not found\n", dname);
            else if (find_file(di, fname) != -1)
                printf("File %s already exists in %s\n", fname, dname);
            else {
                strcpy(d[di].files[d[di].fcount++], fname);
                printf("File %s created in %s\n", fname, dname);
            }
            break;
        case 3:
```

```
            printf("Enter directory name and file name: ");
            scanf("%s %s", dname, fname);
            di = find_dir(dname);
            fi = (di == -1) ? -1 : find_file(di, fname);
            if (fi == -1)
                printf("File %s not found in %s\n", fname, dname);
            else {
                for (i = fi; i < d[di].fcount - 1; i++) strcpy(d[di].files[i],
d[di].files[i + 1]);
                d[di].fcount--;
                printf("File %s deleted from %s\n", fname, dname);
            }
            break;
        case 4:
            printf("Enter directory name and file name: ");
            scanf("%s %s", dname, fname);
            di = find_dir(dname);
            fi = (di == -1) ? -1 : find_file(di, fname);
            if (fi == -1)
                printf("File %s not found in %s\n", fname, dname);
            else
                printf("File %s found in directory %s\n", fname, dname);
            break;
        case 5:
            if (dcount == 0)
                printf("No directories\n");
            else {
                printf("\nDirectory\tFiles\n");
                for (i = 0; i < dcount; i++) {
                    printf("%s\t\t", d[i].dname);
                    for (j = 0; j < d[i].fcount; j++) printf("%s  ",
d[i].files[j]);
                    printf("\n");
                }
            }
            break;
        case 6:
            break;
        default:
            printf("Invalid choice\n");
        }
    } while (choice != 6);
    return 0;
}
```

Output Screenshot:

